

字符串分割

题目描述

给定一个非空字符串S，其被N个'-'分隔成N+1的子串，给定正整数K，要求除第一个子串外，其余的子串每K个字符组成新的子串，并用'-'分隔。对于新组成的每一个子串，如果它含有的小写字母比大写字母多，则将这个子串的所有 大写字母转换为小写 字母；反之，如果它含有的大写字母比小写字母多，则将这个子串的所有小写字母转换为大写字母；大小写字母的数量相等时，不做转换。

输入描述

输入为两行，第一行为参数K，第二行为字符串S。

输出描述

输出转换后的字符串。

用例

输入	3 12abc-abCABc-4aB@
输出	12abc-abc-ABC-4aB-@
说明	子串为12abc、abCABc、4aB@，第一个子串保留，后面的子串每3个字符一组为abC、ABc、4aB、@，abC中小写字母较多，转换为abc，ABc中大写字母较多，转换为ABC，4aB中大小写字母都为1个，不做转换，@中没有字母，连起来即12abc-abc-ABC-4aB-@

输入	12 12abc-abCABc-4aB@
输出	12abc-abCABc4aB@
说明	子串为12abc、abCABc、4aB@，第一个子串保留，后面的子串每12个字符一组为abCABc4aB@，这个子串中大小写字母都为4个，不做转换，连起来即12abc-abCABc4aB@

题目解析

此题应该就是考察字符串的操作能力的，只要熟练掌握 **字符串操作** 即可。

本题比较困惑的是，第一个子串是否要做 **大小写转换** ？看用例说明的意思，应该是不用做的。

下面代码中，我把大小写转换抽成一个方法，考试时可以根据自行使用。

Java算法源码

```
1 import java.util.Scanner;
2 import java.util.StringJoiner;
3
4 public class Main {
5     public static void main(String[] args) {
6         Scanner sc = new Scanner(System.in);
7
8         int k = sc.nextInt();
9         String s = sc.next();
10
11         System.out.println(getResult(k, s));
12     }
13
14     public static String getResult(int k, String s) {
15         String[] arr = s.split("-");
16     }
```

```

17 // 第一个子串不做处理
18     StringJoiner sj = new StringJoiner("-");
19 //     sj.add(convert(arr[0])); // 看用例说明，对应第一个子串是不需要做大小写转换的，但是也拿不准，考试时可以都试下
20 sj.add(arr[0]);
21
22 // 剩余子串重新合并为一个新字符串，每k个字符一组
23 StringBuilder sb = new StringBuilder();
24 for (int i = 1; i < arr.length; i++) sb.append(arr[i]);
25 String newStr = sb.toString();
26
27 for (int i = 0; i < newStr.length(); i += k) {
28     String subStr = newStr.substring(i, Math.min(i + k, newStr.length()));
29     // 子串中小写字母居多，则整体转为小写字母，大写字母居多，则整体转为大写字母
30     sj.add(convert(subStr));
31 }
32
33 return sj.toString();
34 }
35
36 public static String convert(String str) {
37     int lowerCount = 0;
38     int upperCount = 0;
39
40     for (int i = 0; i < str.length(); i++) {
41         char c = str.charAt(i);
42         if (c >= 'a' && c <= 'z') lowerCount++;
43         else if (c >= 'A' && c <= 'Z') upperCount++;
44     }
45
46     if (lowerCount > upperCount) return str.toLowerCase();
47     else if (lowerCount < upperCount) return str.toUpperCase();
48     else return str;
49 }
50 }

```

```
1  /* JavaScript Node ACM模式 控制台输入获取 */
2  const readline = require("readline");
3
4  const rl = readline.createInterface({
5    input: process.stdin,
6    output: process.stdout,
7  });
8
9  const lines = [];
10 rl.on("line", (line) => {
11   lines.push(line);
12
13   if (lines.length === 2) {
14     let k = parseInt(lines[0]);
15     let s = lines[1];
16     console.log(getResult(s, k));
17     lines.length = 0;
18   }
19 });
20
21 function getResult(s, k) {
22   const arr = s.split("-");
23
24   const ans = [];
25   // ans.push(convert(arr[0])); // 看用例说明，对应第一个子串是不需要做大小写转换的，但是也拿不准，考试时可以都试下
26   ans.push(arr[0]);
27
28   // 剩余子串重新合并为一个新字符串，每k个字符一组
29   const newStr = arr.slice(1).join("");
30   for (let i = 0; i < newStr.length; i += k) {
31     const subStr = newStr.slice(i, i + k);
32     // 子串中小写字母居多，则整体转为小写字母，大写字母居多，则整体转为大写字母
33     ans.push(convert(subStr));
34   }
35
36   return ans.join("-");
```

```

37 | }38 |
39 | function convert(s) {
40 |     let lowerCount = 0;
41 |     let upperCount = 0;
42 |
43 |     for (let i = 0; i < s.length; i++) {
44 |         if (s[i] >= "a" && s[i] <= "z") lowerCount++;
45 |         else if (s[i] >= "A" && s[i] <= "Z") upperCount++;
46 |     }
47 |
48 |     if (lowerCount > upperCount) return s.toLowerCase();
49 |     else if (lowerCount < upperCount) return s.toUpperCase();
50 |     else return s;
51 | }

```

Python算法源码

```

1 | # 输入获取
2 | k = int(input())
3 | s = input()
4 |
5 |
6 | def convert(s):
7 |     lowerCount = 0
8 |     upperCount = 0
9 |
10 |     for c in s:
11 |         if 'z' >= c >= 'a':
12 |             lowerCount += 1
13 |         elif 'Z' >= c >= 'A':
14 |             upperCount += 1
15 |
16 |     if lowerCount > upperCount:

```

```

17         return s.lower()18     elif lowerCount < upperCount:
19         return s.upper()
20     else:
21         return s
22
23
24 # 算法入口
25 def getResult():
26     arr = s.split("-")
27
28     ans = []
29     # ans.append(convert(arr[0])) # 看用例说明，对应第一个子串是不需要做大小写转换的，但是也拿不准，考试时可以都试下
30     ans.append(arr[0])
31
32     # 剩余子串重新合并为一个新字符串，每k个字符一组
33     newStr = "".join(arr[1:])
34     for i in range(0, len(newStr), k):
35         subStr = newStr[i:i + k]
36         # 子串中小写字母居多，则整体转为小写字母，大写字母居多，则整体转为大写字母
37         ans.append(convert(subStr))
38
39     return "-".join(ans)
40
41
42 # 算法调用
43 print(getResult())

```

C算法源码

```

1 #include <stdio.h>
2 #include <string.h>
3
4 void toLowerCase(char *s) {
5     int i = 0;
6     while (s[i] != '\0') {

```

运行

```
7         if (s[i] >= 'A' && s[i] <= 'Z') {
8             s[i] += 32;
9         }
10        i++;
11    }
12 }
13
14 void toUpperCase(char *s) {
15     int i = 0;
16     while (s[i] != '\0') {
17         if (s[i] >= 'a' && s[i] <= 'z') {
18             s[i] -= 32;
19         }
20         i++;
21     }
22 }
23
24 void convert(char *s) {
25     int lowerCount = 0;
26     int upperCount = 0;
27
28     int i = 0;
29     while (s[i] != '\0') {
30         char c = s[i];
31
32         if (c >= 'a' && c <= 'z') lowerCount++;
33         else if (c >= 'A' && c <= 'Z') upperCount++;
34
35         i++;
36     }
37
38     if (lowerCount > upperCount) {
39         toLowerCase(s);
40     } else if (lowerCount < upperCount) {
41         toUpperCase(s);
42     }
43 }
```

```

44
45 | int main() {
46     int k;
47     scanf("%d", &k);
48
49     char str[1000] = {'\0'};
50     scanf("%s", str);
51
52     char res[1000] = {'\0'};
53     char newStr[1000] = {'\0'};
54
55     // 如果第二行输入的开头是 '-', 则说明第一个字串是空串
56     int first = str[0] != '-';
57
58     char *token = strtok(str, "-");
59     while (token != NULL) {
60
61         if (first) {
62             // 第一个子串不做处理
63             // 看用例说明, 对应第一个子串是不需要做大小写转换的, 但是也拿不准, 考试时可以都试下
64             strcat(res, token);
65             first = 0;
66         } else {
67             // 剩余子串重新合并为一个新字符串, 每k个字符一组
68             strcat(newStr, token);
69         }
70
71         token = strtok(NULL, "-");
72     }
73
74     for (int i = 0; i < strlen(newStr); i += k) {
75         // 新字符串每k个字符一组
76         char dest[k + 1];
77         strncpy(dest, newStr + i, k);
78         dest[k] = '\0';
79
80         // 子串中小写字母居多, 则整体转为小写字母, 大写字母居多, 则整体转为大写字母

```



```

81         convert(dest);
82     }
83     strcat(res, "-");
84     strcat(res, dest);
85 }
86
87 puts(res);
88
89 return 0;
90 }

```

C++算法源码

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  void convert(string& s) {
5      int lowerCount = 0;
6      int upperCount = 0;
7
8      for (const auto &c: s) {
9          if (c >= 'a' && c <= 'z') lowerCount++;
10         else if (c >= 'A' && c <= 'Z') upperCount++;
11     }
12
13     if (lowerCount > upperCount) {
14         transform(s.begin(), s.end(), s.begin(), [](char c) {
15             return tolower(c);
16         });
17     } else if (lowerCount < upperCount) {
18         transform(s.begin(), s.end(), s.begin(), [](char c) {
19             return toupper(c);
20         });
21     }

```

```

22 }23 |
24 int main() {
25     int k;
26     cin >> k;
27
28     string str;
29     cin >> str;
30
31     stringstream ss(str);
32     string token;
33
34     string res;
35     string newStr;
36
37     // 如果第二行输入的开头是 '-', 则说明第一个字串是空串
38     bool first = str[0] != '-';
39
40     while (getline(ss, token, '-')) {
41         if (first) {
42             // 第一个子串不做处理
43             // 看用例说明, 对应第一个子串是不需要做大小写转换的, 但是也拿不准, 考试时可以都试下
44             res += token;
45             first = false;
46         } else {
47             // 剩余子串重新合并为一个新字符串, 每k个字符一组
48             newStr += token;
49         }
50     }
51
52     for (int i = 0; i < newStr.size(); i += k) {
53         // 新字符串每k个字符一组
54         string dest = newStr.substr(i, k);
55
56         // 子串中小写字母居多, 则整体转为小写字母, 大写字母居多, 则整体转为大写字母
57         convert(dest);
58
59         res += "-" + dest;

```

```
60 | }  
    | 61 |  
62 | cout << res << endl;  
63 |  
64 | return 0;  
65 | }
```